

Python Fundamentals

- Python Introduction
- Input and Output in python
- Python Variables
- Python Operators
- Python Keywords
- Python Data Types
- Conditional statements in python
- Loops in Python - For, While and Nested Loops
- Python Functions
- Recursion in Python
- Python Lambda functions

1. Python Introduction

Python is a high-level, interpreted programming language that was created in 1991 by Guido van Rossum. Its main goal was to make code simple, readable, and easy to learn.

Python has a very clean and user-friendly syntax, which makes it easy for beginners to learn. This language supports object-oriented, procedural, and functional programming.

Python is used in many fields such as:

- Web development (Django, Flask)
- Data science aur Machine Learning (NumPy, Pandas, TensorFlow)
- Automation aur Scripting
- Artificial Intelligence
- Game aur App development
- In short, Python ek powerful aur versatile language hai jo har tarah ke projects ke liye use ki ja sakti hai.

```
In [14]: # Printing hello world
```

```
print('hello world')
```

```
hello world
```

2. Input and Output in python

Understanding input and output operations is fundamental to Python programming. With the `print()` function, we can display output in various formats, while the `input()` function enables interaction with users by gathering input during program execution.

Understanding input and output operations in Python is very important.

- The **`print()`** function is used to display output — meaning we can show messages, numbers, or results on the screen.
- The **`input()`** function is used to take data from the user — whatever the user types while the program is running is stored as input.

Taking input in Python

Python's `input()` function is used to take user input. By default, it returns the user input in form of a string.

```
In [15]: name = input('enter your name: ')
print('hello',name,'how are you')
```

```
enter your name: kamran
hello kamran how are you
```

Printing Variables

- A variable in Python is a name used to store a value.
- It acts like a container that holds data (like numbers or text).
- You don't need to declare its type — Python decides it automatically.
- For example: `x = 10` means variable `x` stores the value 10.

```
In [16]: # Single variable
s = "Bob"
print(s)

# Multiple Variables
s = "Alice"
age = 25
city = "New York"
print(s, age, city)
```

```
Bob
Alice 25 New York
```

Take Multiple Input in Python

We are taking multiple input from the user in a single line, splitting the values entered by the user into separate variables for each value using the `split()` method. Then, it prints the values with corresponding labels, either two or three, based on the number of inputs provided by the user.

```
In [17]: # taking two inputs at a time
x, y = input("Enter two values: ").split()
print("Number of boys: ", x)
print("Number of girls: ", y)

# taking three inputs at a time
x, y, z = input("Enter three values: ").split()
print("Total number of students: ", x)
print("Number of boys is : ", y)
print("Number of girls is : ", z)
```

```
Enter two values: 10 15
Number of boys:  10
Number of girls: 15
Enter three values: 15 20 25
Total number of students: 15
Number of boys is : 20
Number of girls is : 25
```

How to Change the Type of Input in Python

By default input() function helps in taking user input as string. If any user wants to take input as int or float, we just need to typecast it.

Print Names in Python

The code prompts the user to input a string (the color of a rose), assigns it to the variable color and then prints the inputted color.

```
In [1]: # Taking input as string
color = input('What color is rose? :')
print(color)
```

```
What color is rose? :Red
Red
```

Print Numbers in Python

The code prompts the user to input an integer representing the number of roses, converts the input to an integer using typecasting and then prints the integer value.

```
In [6]: # Taking input as int
# Typecasting to int

n= int(input('How many roses? :'))
print('There are',n,'roses')
```

```
How many roses? :12
There are 12 roses
```

Print Float/Decimal Number in Python

The code prompts the user to input the price of each rose as a floating-point number, converts the input to a float using typecasting and then prints the price.

```
In [7]: # Taking input as float
# Typecasting to float

price = float(input('Price of each rose?: '))
print(price)
```

```
Price of each rose?: 55.34
55.34
```

Find DataType of Input in Python

A **data type** tells us what kind of data is stored in a variable (such as numbers, text, lists, etc.).

In the given example, we are printing the type of variable x. We will determine the type of an object in Python.

```
In [8]: a = "Hello World"
b = 10
c = 11.22
d = ("Geeks", "for", "Geeks")
e = ["Geeks", "for", "Geeks"]
f = {"Geeks": 1, "for":2, "Geeks":3}
```

```
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'tuple'>
<class 'list'>
<class 'dict'>
```

3.Python Variables

In Python, variables are used to store data that can be referenced and manipulated during program execution. A variable is essentially a name that is assigned to a value.

```
In [9]: # Variable 'x' stores the integer value 5
x = 5

# Variable 'name' stores the string "Samantha"
name = "Samantha"

print(x)
print(name)
```

```
5
Samantha
```

Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

- Variable names can only contain letters, digits and underscores (_).
- A variable name cannot start with a digit.
- Variable names are case-sensitive (myVar and myvar are different).
- Avoid using Python keywords (e.g., if, else, for) as variable names.

Valid :

```
In [1]: age = 21
_colour = "lilac"
total_score = 90
```

Invalid :

```
In [ ]: 1name = "Error" # Starts with a digit
        class = 10     # 'class' is a reserved keyword
        user-name = "Doe" # Contains a hyphen
```

Assigning Values to Variables

Basic Assignment

Variables in Python are assigned values using the = operator.

```
In [ ]: x = 5
        y = 3.14
        z = "Hi"
```

Dynamic Typing

Python variables are dynamically typed, meaning the same variable can hold different types of values during execution.

```
In [ ]: x = 10
        x = "Now a string"
```

Multiple Assignments

Python allows multiple variables to be assigned values in a single line.

Assigning the Same Value

Python allows assigning the same value to multiple variables in a single line, which can be useful for initializing variables with the same value.

```
In [10]: a = b = c = 100
        print(a, b, c)
```

100 100 100

Assigning Different Values

We can assign different values to multiple variables simultaneously, making the code concise and easier to read.

```
In [11]: x, y, z = 1, 2.5, "Python"
        print(x, y, z)
```

1 2.5 Python

Type Casting a Variable

Type casting refers to the process of converting the value of one data type into another. Python provides several built-in functions to facilitate casting, including `int()`, `float()` and `str()` among others.

Basic Casting Functions

- `int()` - Converts compatible values to an integer.
- `float()` - Transforms values into floating-point numbers.
- `str()` - Converts any data type into a string.

```
In [12]: # Casting variables
s = "10" # Initially a string
n = int(s) # Cast string to integer
cnt = 5
f = float(cnt) # Cast integer to float
age = 25
s2 = str(age) # Cast integer to string

# Display results
print(n)
print(f)
print(s2)
```

```
10
5.0
25
```

Getting the Type of Variable

In Python, we can determine the type of a variable using the `type()` function. This built-in function returns the type of the object passed to it.

```
In [13]: # Define variables with different data types
n = 42
f = 3.14
s = "Hello, World!"
li = [1, 2, 3]
d = {'key': 'value'}
bool = True

# Get and print the type of each variable
print(type(n))
print(type(f))
print(type(s))
print(type(li))
print(type(d))
print(type(bool))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'dict'>
<class 'bool'>
```

Delete a Variable Using `del` Keyword

We can remove a variable from the namespace using the `del` keyword. This effectively deletes the variable and frees up the memory it was using.

```
In [14]: # Assigning value to variable
x = 10
print(x)

# Removing the variable using del
del x

# Trying to print x after deletion will raise an error
# print(x) # Uncommenting this line will raise NameError: name 'x' is not defined
```

10

Practical Examples

1. Swapping Two Variables

Using multiple assignments, we can swap the values of two variables without needing a temporary variable.

```
In [16]: a, b = 5, 10
a, b = b, a
print(a,b)
print('value of a is',a)
```

10 5
value of a is 10

2. Counting Characters in a String

Assign the results of multiple operations on a string to variables in one line.

```
In [17]: word = "Python"
length = len(word)
print("Length of the word:", length)
```

Length of the word: 6

4. Python Operators

In Python programming, Operators in general are used to perform operations on values and variables.

- Operators: Special symbols like -, +, *, /, etc.
- Operands: Value on which the operator is applied.

Types of Operators in Python

Operators in Python

Operators	Type
+, -, *, /, %	Arithmetic operator
<, <=, >, >=, ==, !=	Relational operator
AND, OR, NOT	Logical operator
&, , <<, >>, -, ^	Bitwise operator
=, +=, -=, *=, %=	Assignment operator

Arithmetic Operators

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

In Python 3.x the result of division is a floating-point while in Python 2.x division of 2 integers was an integer. To obtain an integer result in Python 3.x floored (`//` integer) is used.

```
In [18]: # Variables
a = 15
b = 4

# Addition
print("Addition:", a + b)

# Subtraction
print("Subtraction:", a - b)

# Multiplication
print("Multiplication:", a * b)

# Division
print("Division:", a / b)

# Floor Division
print("Floor Division:", a // b)

# Modulus
print("Modulus:", a % b)

# Exponentiation
print("Exponentiation:", a ** b)
```

```
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Floor Division: 3
Modulus: 3
Exponentiation: 50625
```

Logical Operators

Python Logical operators perform Logical AND, Logical OR and Logical NOT operations. It is used to combine conditional statements.

The precedence of Logical Operators in Python is as follows:

1. Logical and
2. logical or
3. logical nor

```
In [19]: a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

```
False
True
False
```

Bitwise Operators

Python Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Bitwise Operators in Python are as follows:

- Bitwise NOT
- Bitwise Shift
- Bitwise AND
- Bitwise XOR
- Bitwise OR

```
In [1]: a = 10
b = 4

print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

```
0
14
-11
14
2
40
```

Assignment Operators

Python Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand.

```
In [21]: a = 10
b = a
print(b)
b += a
print(b)
b -= a
print(b)
b *= a
print(b)
b <<= a
print(b)
```

```
10
20
10
100
102400
```

Identity Operators

In Python, `is` and `is not` are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

1. `is` is True if the operands are identical
2. `is not` is True if the operands are not identical

```
In [20]: a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

```
True
True
```

Membership Operators

In Python, `in` and `not in` are the membership operators that are used to test whether a value or variable is in a sequence.

1. `in` is True if value is found in the sequence
2. `not in` is True if value is not found in the sequence

```
In [22]: x = 24
y = 20
list = [10, 20, 30, 40, 50]

if (x not in list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

```
x is NOT present in given list
y is present in given list
```

Ternary Operator

In Python, Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.

It simply allows testing a condition in a single line replacing the multiline if-else, making the code compact.

Syntax : `[on_true] if [expression] else [on_false]`

```
In [23]: a, b = 10, 20
min = a if a < b else b

print(min)
```

```
10
```

Precedence and Associativity of Operators

In Python, Operator precedence and associativity determine the priorities of the operator.

Operator Precedence

This is used in an expression with more than one operator with different precedence to determine which operation to perform first.

```
In [24]: expr = 10 + 20 * 30
print(expr)
name = "Alex"
age = 0

if name == "Alex" or name == "John" and age >= 2:
    print("Hello! Welcome.")
else:
    print("Good Bye!!")
```

```
610
Hello! Welcome.
```

Operator Associativity

If an expression contains two or more operators with the same precedence then Operator Associativity is

```
In [25]: print(100 / 10 * 10)
print(5 - 2 + 3)
print(5 - (2 + 3))
print(2 ** 3 ** 2)
```

```
100.0
6
0
512
```

5. Python Keywords

Keywords in Python are special reserved words that are part of the language itself. They define the rules and structure of Python programs, which means you cannot use them as names for your variables, functions, classes, or any other identifiers.

Getting List of all Python keywords

We can also get all the keyword names using the below code.

```
In [26]: import keyword

# printing all keywords at once using "kwlist()"
print("The list of keywords is : ")
print(keyword.kwlist)
```

```
The list of keywords is :
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'globa
l', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'r
eturn', 'try', 'while', 'with', 'yield']
```

How to Identify Python Keywords ?

- With Syntax Highlighting - Most of IDEs provide syntax-highlight feature. You can see Keywords appearing in different color or style.
- Look for SyntaxError - This error will encounter if you have used any keyword incorrectly. Note that keywords can not be used as identifiers (variable or a function name).

What Happens if We Use Keywords as Variable Names ?

If we attempt to use a keyword as a variable, Python will raise a **SyntaxError**. Let's look at an example

```
In [27]: for = 10
print(for)
```

```
Cell In[27], line 1
```

```
for = 10
^
```

```
SyntaxError: invalid syntax
```

Let's categorize all keywords based on context for a more clear understanding.

Category	Keywords
Value Keywords	True , False , None
Operator Keywords	and , or , not , is , in
Control Flow Keywords	if , else , elif , for , while , break , continue , pass , try , except , finally , raise , assert
Function and Class	def , return , lambda , yield , class
Context Management	with , as
Import and Module	import , from
Scope and Namespace	global , nonlocal
Async Programming	async , await

Keywords vs Identifiers

Keywords	Identifiers
Reserved words in Python that have a specific meaning.	Names given to variables, functions, classes, etc.
Cannot be used as variable names.	Can be used as variable names (if not a keyword).
Examples: if, else, for, while	Examples: x, number, sum, result
Part of the Python syntax.	User-defined, meaningful names in the code.
They cannot be redefined or changed.	Can be defined and redefined by the programmer.

Variable and Keywords

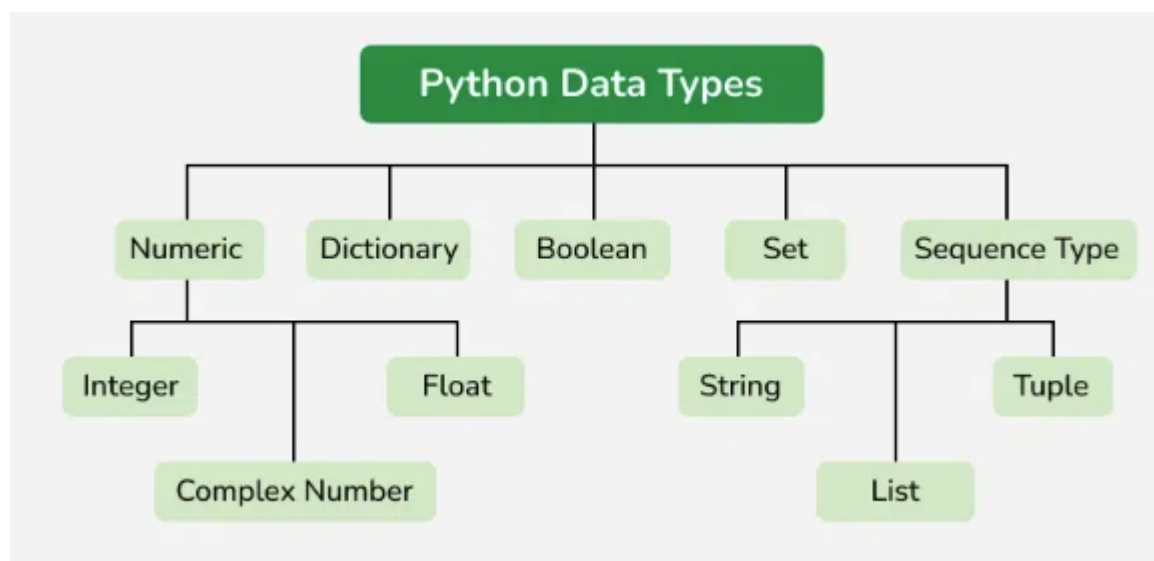
Variables	Keywords
Used to store data.	Reserved words with predefined meanings in Python.
Can be created, modified, and deleted by the programmer.	Cannot be modified or used as variable names.
Examples: x, age, name	Examples: if, while, for
Hold values that are manipulated in the program.	Used to define the structure of Python code.
Variable names must follow naming rules but are otherwise flexible.	Fixed by Python language and cannot be altered.

6. Python Data Types

Data types in Python are a way to classify data items. They represent the kind of value, which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.

The following are standard or built-in data types in Python:

- Numeric: int, float, complex
- Sequence Type: string, list, tuple
- Mapping Type: dict
- Boolean: bool
- Set Type: set, frozenset
- Binary Types: bytes, bytearray, memoryview



Below code assigns variable 'x' different values of few Python data types - int, float, list, tuple and string. Each assignment replaces previous value, making 'x' take on data type and value of most recent assignment.

```
In [28]: x = 50 # int
x = 60.5 # float
x = "Hello World" # string
x = ["geeks", "for", "geeks"] # list
x = ("geeks", "for", "geeks") # tuple
```

1. Numeric Data Types

Python numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- **Integers:** value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- **Float:** value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- **Complex Numbers:** It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - 2+3j

```
In [29]: a = 5
print(type(a))

b = 5.0
print(type(b))

c = 2 + 4j
print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion. There are several sequence data types of Python:

String Data Type

Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

```
In [30]: s = 'Welcome to the Geeks World'
print(s)

# check data type
print(type(s))

# access string with index
print(s[1])
print(s[2])
print(s[-1])
```

```
Welcome to the Geeks World
<class 'str'>
e
l
d
```

List Data Type

Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items. It is very flexible as items in a list do not need to be of the same type.

Creating a List in Python

Lists in Python can be created by just placing sequence inside the square brackets[].

```
In [31]: # Empty List
a = []

# List with int values
a = [1, 2, 3]
print(a)

# List with mixed values int and String
b = ["Geeks", "For", "Geeks", 4, 5]
print(b)
```

```
[1, 2, 3]
['Geeks', 'For', 'Geeks', 4, 5]
```

Access List Items

In order to access the list items refer to index number. In Python, negative sequence indexes represent positions from end of the array. Instead of having to compute offset as in `List[len(List)-3]`, it is enough to just write `List[-3]`. Negative indexing means beginning from end, -1 refers to last item, -2 refers to second-last item, etc.

```
In [ ]: a = ["Geeks", "For", "Geeks"]
print("Accessing element from the list")
print(a[0])
print(a[2])

print("Accessing element using negative indexing")
print(a[-1])
print(a[-3])
```

Tuple Data Type

Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

Creating a Tuple in Python

In Python, tuples are created by placing a sequence of values separated by a 'comma' with or without the use of parentheses for grouping data sequence. Tuples can contain any number of elements and of any datatype (like strings, integers, lists, etc.).

Note: Tuples can also be created with a single element, but it is a bit tricky. Having one element in the parentheses is not sufficient, there must be a trailing '**comma**' to make it a tuple.

```
In [32]: # initiate empty tuple
tup1 = ()

tup2 = ('Geeks', 'For')
print("\nTuple with the use of String: ", tup2)
```

```
Tuple with the use of String: ('Geeks', 'For')
```

Note - The creation of a Python tuple without the use of parentheses is known as Tuple Packing.

Access Tuple Items

In order to access tuple items refer to the index number. Use the index operator [] to access an item in a

```
In [33]: tup1 = (1, 2, 3, 4, 5)
```

```
# access tuple items
print(tup1[0])
print(tup1[-1])
print(tup1[-3])
```

```
1
5
3
```

3. Boolean Data Type

Python Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

Python ka Boolean data type do built-in values deta hai — True aur False. Agar koi object True ke equal hota hai, to usse truthy kehte hain (yaani true maana jata hai), aur agar False ke equal hota hai, to usse falsy kehte hain (yaani false maana jata hai).

Lekin sirf Boolean hi nahi — Python me non-Boolean objects (jaise numbers, strings, lists, etc.) bhi Boolean context me check kiye ja sakte hain. For example:

0, "" (empty string), [] (empty list) → False hote hain.

1, "Hello", [10, 20] → True hote hain.

Boolean type ko Python me bool class se represent kiya jata hai.

```
In [34]: print(type(True))
print(type(False))
print(type(true))
```

```
<class 'bool'>
<class 'bool'>
```

NameError

Traceback (most recent call last)

```
Cell In[34], line 3
      1 print(type(True))
      2 print(type(False))
----> 3 print(type(true))
```

NameError: name 'true' is not defined

Truthy and Falsy Values

In Python, truthy and falsy values are values that evaluate to True or False in a Boolean context. Truthy values behave like True, while falsy values behave like False when used in conditions.

```
In [35]: if 1:
          print("1 is truthy")

          if not 0:
              print("0 is falsy")
```

```
1 is truthy
0 is falsy
```

4. Set Data Type

In Python Data Types, Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

Create a Set in Python

Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'. The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

```
In [36]: # initializing empty set
s1 = set()

s1 = set("GeeksForGeeks")
print("Set with the use of String: ", s1)

s2 = set(["Geeks", "For", "Geeks"])
print("Set with the use of List: ", s2)
```

```
Set with the use of String: {'o', 'F', 'G', 'e', 's', 'r', 'k'}
Set with the use of List: {'For', 'Geeks'}
```

Access Set Items

Set items cannot be accessed by referring to an index, since sets are unordered the items have no index. But we can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the keyword in.

```
In [7]: set1 = set(["Geeks", "For", "Geeks"]) #Duplicates are removed automatically
print(set1)

# Loop through set
for i in set1:
    print(i, end=" ") #prints elements one by one

# check if item exist in set
print("Geeks" in set1)
```

```
{'Geeks', 'For'}
Geeks For True
```

5. Dictionary Data Type

A dictionary in Python is a collection of data values, used to store data values like a map, unlike other Python Data Types, a Dictionary holds a key: value pair. Key-value is provided in dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

Create a Dictionary in Python

Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable. The dictionary can also be created by the built-in function dict().

Note - Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

```
In [37]: # initialize empty dictionary
d = {}

d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d)

# creating dictionary using dict() constructor
d1 = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})
print(d1)

{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

Accessing Key-value in Dictionary

In order to access items of a dictionary refer to its key name. Key can be used inside square brackets. Using get() method we can access dictionary elements.

```
In [38]: d = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# Accessing an element using key
print(d['name'])

# Accessing a element using get
print(d.get(3))
```

For
Geeks

7.Conditional Statements in Python

Conditional statements in Python are used to execute certain blocks of code based on specific conditions. These statements help control the flow of a program, making it behave differently in different situations.

If Conditional Statement

If statement is the simplest form of a conditional statement. It executes a block of code if the given condition is true.

```
In [39]: age = 20
if age >= 18:
    print("Eligible to vote.")
```

Eligible to vote.

Short Hand if

Short-hand if statement allows us to write a single-line if statement.

```
In [40]: age = 19
if age > 18: print("Eligible to Vote.")
```

Eligible to Vote.

This is a compact way to write an if statement. It executes print statement if the condition is true.

If else Conditional Statement

If Else allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False. Else block provides a way to handle all other cases that don't meet the specified conditions.

```
In [41]: age = 10
if age <= 12:
    print("Travel for free.")
else:
    print("Pay for ticket.")
```

Travel for free.

Short Hand if-else

The short-hand if-else statement allows us to write a single-line if-else statement.

```
In [42]: marks = 45
res = "Pass" if marks >= 40 else "Fail"
print(f"Result: {res}")
```

Result: Pass

Note: This method is also known as ternary operator. Ternary Operator essentially a shorthand for the if-else statement that allows us to write more compact and readable code, especially for simple conditions.

elif Statement

elif statement in Python stands for "else if." It allows us to check multiple conditions, providing a way to execute different blocks of code based on which condition is true. Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.

```
In [43]: age = 25

if age <= 12:
    print("Child.")
elif age <= 19:
    print("Teenager.")
elif age <= 35:
    print("Young adult.")
else:
    print("Adult.")
```

Young adult.

The code checks the value of age using if-elif-else. Since age is 25, it skips the first two conditions (age <= 12 and age <= 19), and the third condition (age <= 35) is True, so it prints "Young adult."

Nested if..else Conditional Statement

Nested if..else means an if-else statement inside another if statement. We can use nested if statements to check conditions within conditions.

```
In [44]: age = 70
is_member = True

if age >= 60:
    if is_member:
        print("30% senior discount!")
    else:
        print("20% senior discount.")
else:
    print("Not eligible for a senior discount.")
```

30% senior discount!

Ternary Conditional Statement

A ternary conditional statement is a compact way to write an if-else condition in a single line. It's sometimes called a "conditional expression."

```
In [45]: # Assign a value based on a condition
age = 20
s = "Adult" if age >= 18 else "Minor"
print(s)
```

Adult

Here:

- If age >= 18 is True, status is assigned "Adult".
- Otherwise, status is assigned "Minor".

Match-Case Statement

match-case statement is Python's version of a switch-case found in other languages. It allows us to match a variable's value against a set of patterns.

```
In [46]: number = 2

match number:
    case 1:
        print("One")
    case 2 | 3:
        print("Two or Three")
    case _:
        print("Other number")
```

Two or Three

8.Loops in Python - For, While and Nested Loops

Last Updated : 04 Oct, 2025 Loops in Python are used to repeat actions efficiently. The main types are For loops (counting through items) and While loops (based on conditions).

For Loop

For loops is used to iterate over a sequence such as a list, tuple, string or range. It allow to execute a block of code repeatedly once for each item in the sequence

```
In [47]: n = 4
for i in range(0, n):
    print(i)
```

```
0
1
2
3
```

Explanation: This code prints the numbers from 0 to 3 (inclusive) using a for loop that iterates over a range from 0 to n-1 (where n = 4).

Iterating Over List, Tuple, String and Dictionary Using for Loops in Python

```
In [48]: li = ["geeks", "for", "geeks"]
for x in li:
    print(x)

tup = ("geeks", "for", "geeks")
for x in tup:
    print(x)

s = "abc"
for x in s:
    print(x)

d = dict({'x':123, 'y':354})
for x in d:
    print("%s %d" % (x, d[x]))

set1 = {10, 30, 20}
for x in set1:
    print(x),
```

```
geeks
for
geeks
geeks
for
geeks
a
b
c
x 123
y 354
10
20
30
```

Iterating by Index of Sequences

We can also use the index of elements in the sequence to iterate. The key idea is to first calculate the length of the list and then iterate over the sequence within the range of this length.

```
In [6]: li = ["geeks", "for", "geeks"]  
for index in range(len(li)):  
    print(li[index])
```

```
geeks  
for  
geeks
```

Explanation: This code iterates through each element of the list using its index and prints each element one by one. The `range(len(list))` generates indices from 0 to the length of the list minus 1.

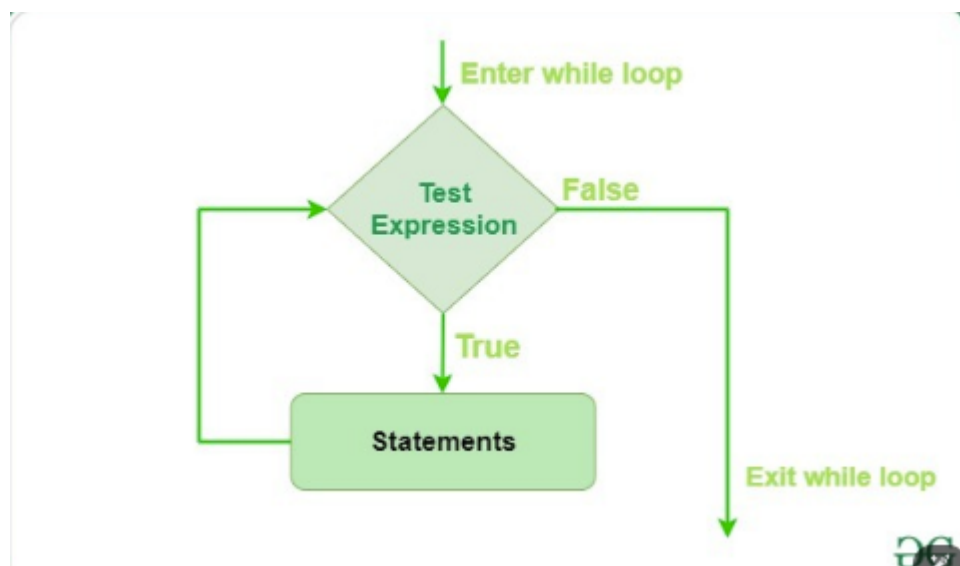
While Loop

In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

In below code, loop runs as long as the condition `cnt < 3` is true. It increments the counter by 1 on each iteration and prints "Hello Geek" three times.

```
In [49]: cnt = 0  
while (cnt < 3):  
    cnt = cnt + 1  
    print("Hello Geek")
```

```
Hello Geek  
Hello Geek  
Hello Geek
```



Infinite While Loop

If we want a block of code to execute infinite number of times then we can use the while loop in Python to do so.

Code given below uses a 'while' loop with the condition **"True"**, which means that the loop will run infinitely until we break out of it using **"break"** keyword or some other logic.

```
In [ ]: while (True):  
        print("Hello Geek")
```

Note: It is suggested not to use this type of loop as it is a never-ending infinite loop where the condition is

Nested Loops

Python programming language allows to use one loop inside another loop which is called nested loop. Following example illustrates the concept.

```
In [52]: from __future__ import print_function
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

```
1
2 2
3 3 3
4 4 4 4
```

Explanation: In the above code we use nested loops to print the value of i multiple times in each row, where the number of times it prints i increases with each iteration of the outer loop. The print() function prints the value of i and moves to the next line after each row.

A final note on loop nesting is that we can put any type of loop inside of any other type of loops in Python. For example, a for loop can be inside a while loop or vice versa.

Loop Control Statements

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue Statement The continue statement in Python returns the control to the beginning of the loop.

```
In [53]: for letter in 'geeksforgeeks':
        if letter == 'e' or letter == 's':
            continue
        print('Current Letter :', letter)
```

```
Current Letter : g
Current Letter : k
Current Letter : f
Current Letter : o
Current Letter : r
Current Letter : g
Current Letter : k
```

Explanation: The continue statement is used to skip the current iteration of a loop and move to the next iteration. It is useful when we want to bypass certain conditions without terminating the loop.

Break Statement

The break statement in Python brings control out of the loop.

```
In [54]: for letter in 'geeksforgeeks':  
        if letter == 'e' or letter == 's':  
            break  
  
print('Current Letter :', letter)
```

Current Letter : e

Explanation: break statement is used to exit the loop prematurely when a specified condition is met. In this example, the loop breaks when the letter is either 'e' or 's', stopping further iteration.

Pass Statement

We use pass statement in Python to write empty loops. Pass is also used for empty control statements, functions and classes.

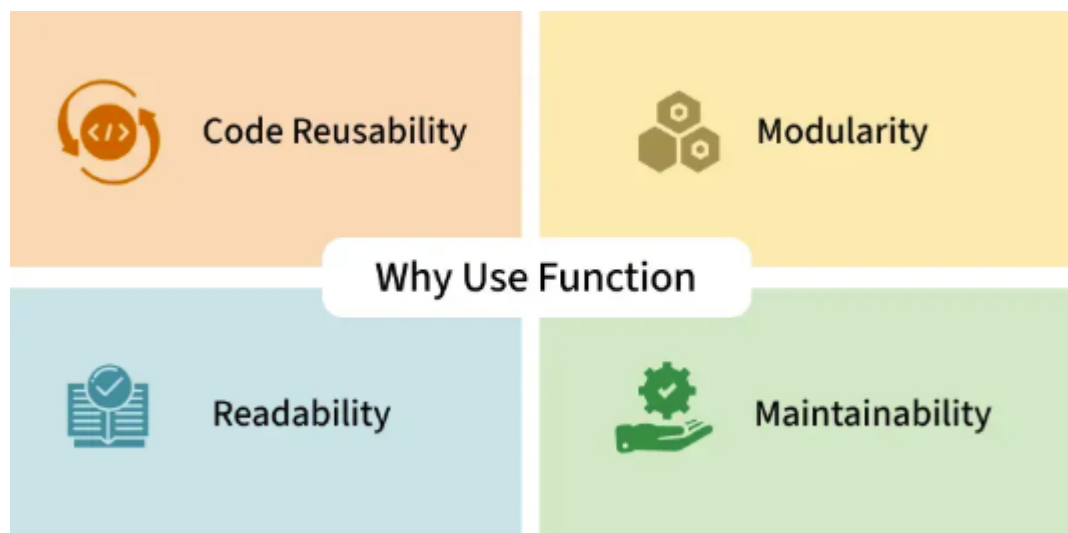
```
In [55]: for letter in 'geeksforgeeks':  
        pass  
print('Last Letter :', letter)
```

Last Letter : s

Explanation: In this example, the loop iterates over each letter in 'geeksforgeeks' but doesn't perform any operation, and after the loop finishes, the last letter ('s') is printed.

9. Python Functions

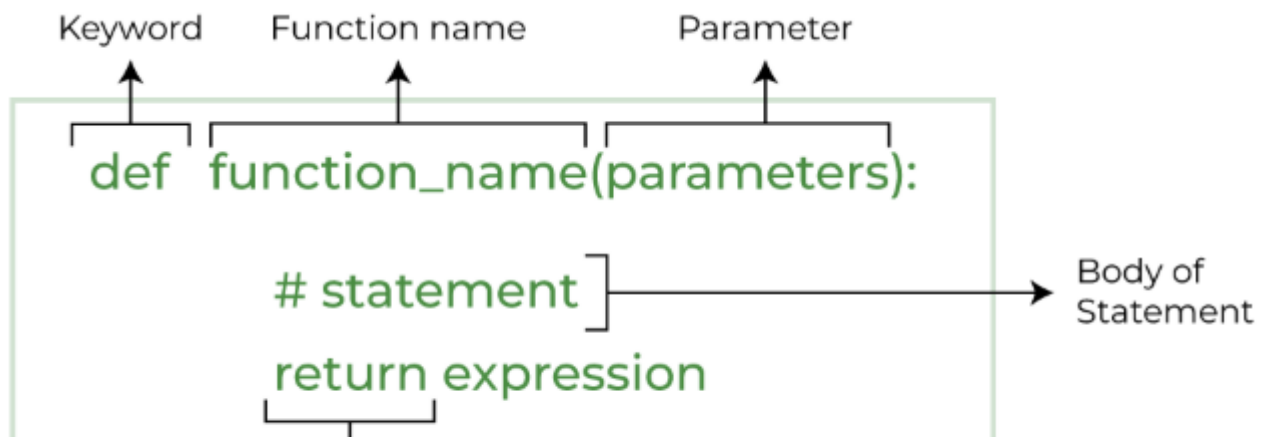
Python Functions are a block of statements that does a specific task. The idea is to put some commonly or repeatedly done task together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.



Defining a Function

We can define a function in Python, using the def keyword. A function might take input in the form of parameters.

The syntax to declare a function is:



```
In [56]: # Here, we define a function using def that prints a welcome message when called.  
def fun():  
    print("Welcome to GFG")
```

Calling a Function

After creating a function in Python we can call it by using the name of the functions followed by parenthesis containing parameters of that particular function.

```
In [57]: def fun():  
        print("Welcome to GFG")  
  
fun() # Driver code to call a function
```

Welcome to GFG

Function Arguments

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

Syntax:

```
def function_name(parameters):  
    """Docstring"""  
    # body of the function  
    return expression
```

We will create a simple function in Python to check whether the number passed as an argument to the function is even or odd.

```
In [58]: def evenOdd(x):  
        if (x % 2 == 0):  
            return "Even"  
        else:  
            return "Odd"  
  
print(evenOdd(16))  
print(evenOdd(7))
```

Even
Odd

Types of Function Arguments

Python supports various types of arguments that can be passed at the time of the function call. In Python, we have the following function argument types in Python, Let's explore them one by one.

1. Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument.

```
In [59]: def myFun(x, y=50):  
        print("x: ", x)  
        print("y: ", y)  
  
        myFun(10)
```

```
x:  10  
y:  50
```

2. Keyword Arguments

In keyword arguments, values are passed by explicitly specifying the parameter names, so the order doesn't matter.

```
In [61]: def student(fname, lname):  
        print(fname, lname)  
  
        student(fname='Geeks', lname='Practice')  
        student(lname='Practice', fname='Geeks')
```

```
Geeks Practice  
Geeks Practice
```

3. Positional Arguments

In positional arguments, values are assigned to parameters based on their order in the function call.

```
In [62]: def nameAge(name, age):  
        print("Hi, I am", name)  
        print("My age is ", age)  
  
        print("Case-1:")  
        nameAge("Suraj", 27)  
  
        print("\nCase-2:")  
        nameAge(27, "Suraj")
```

```
Case-1:  
Hi, I am Suraj  
My age is  27
```

```
Case-2:  
Hi, I am 27  
My age is  Suraj
```

4. Arbitrary Arguments

In Python Arbitrary Keyword Arguments, *args and **kwargs can pass a variable number of arguments to a function using special symbols. There are two special symbols:

*args in Python (Non-Keyword Arguments)

****kwargs** in Python (Keyword Arguments)

This code separately shows non-keyword (*args) and keyword (**kwargs) arguments in the same function.

```
In [63]: def myFun(*args, **kwargs):  
          print("Non-Keyword Arguments (*args):")  
          for arg in args:  
              print(arg)  
  
          print("\nKeyword Arguments (**kwargs):")  
          for key, value in kwargs.items():  
              print(f"{key} == {value}")  
  
          # Function call with both types of arguments  
          myFun('Hey', 'Welcome', first='Geeks', mid='for', last='Geeks')
```

Non-Keyword Arguments (*args):

Hey

Welcome

Keyword Arguments (**kwargs):

first == Geeks

mid == for

last == Geeks

Function within Functions

A function defined inside another function is called an inner function (or nested function). It can access variables from the enclosing function's scope and is often used to keep logic protected and organized.

```
In [5]: def f1():  
          s = 'I love GeeksforGeeks'  
          def f2():  
              print(s)  
  
          f2()  
          f1()
```

I love GeeksforGeeks

Anonymous Functions

In Python, an anonymous function means that a function is without a name. As we already know the def keyword is used to define the normal functions and the lambda keyword is used to create anonymous functions.

```
In [3]: def cube(x): return x*x*x    # without Lambda  
          cube_l = lambda x : x*x*x  # with Lambda  
  
          print(cube(7))  
          print(cube_l(7))
```

343

343

Return Statement in Function

The return statement ends a function and sends a value back to the caller. It can return any data type, multiple values (packed into a tuple), or None if no value is given.

Syntax:

return [expression]

```
In [64]: def square_value(num):  
        """This function returns the square  
        value of the entered number"""  
        return num**2  
  
        print(square_value(2))  
        print(square_value(-4))
```

```
4  
16
```

Pass by Reference and Pass by Value

In Python, variables are references to objects. When we pass them to a function, the behavior depends on whether the object is mutable (like lists, dictionaries) or immutable (like integers, strings, tuples).

- Mutable objects: Changes inside the function affect the original object.
- Immutable objects: The original value remains unchanged.

```
In [65]: # Function modifies the first element of list  
def myFun(x):  
    x[0] = 20  
  
lst = [10, 11, 12, 13]  
myFun(lst)  
print(lst)    # list is modified  
  
# Function tries to modify an integer  
def myFun2(x):  
    x = 20  
  
a = 10  
myFun2(a)  
print(a)      # integer is not modified
```

```
[20, 11, 12, 13]  
10
```

Note: Technically, Python uses "pass-by-object-reference". Mutable objects behave like pass by reference, while immutable objects behave like pass by value

Recursive Functions

A recursive function is a function that calls itself to solve a problem. It is commonly used in mathematical and divide-and-conquer problems. Always include a base case to avoid infinite recursion.

```
In [66]: def factorial(n):  
        if n == 0:  
            return 1  
        else:  
            return n * factorial(n - 1)  
  
print(factorial(4))
```

24

Here we have created a recursive function to calculate the factorial of the number. It calls itself until a base case (n==0) is met.

10.Recursion in Python

Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem by breaking it into smaller, simpler subproblems.

In Python, recursion is especially useful for problems that can be divided into identical smaller tasks, such as mathematical calculations, tree traversals or divide-and-conquer algorithms.

Working of Recursion

A recursive function is just like any other Python function except that it calls itself in its body. Let's see basic structure of recursive function:

```
In [4]: def recursive_function(parameters):  
        if base_case_condition:  
            return base_result  
        else:  
            return recursive_function(modified_parameters)
```

Recursive function contains two key parts:

Base Case: The stopping condition that prevents infinite recursion. Recursive Case: The part of the function where it calls itself with modified parameters.

Examples of Recursion

Let's understand recursion better with the help of some examples.

Example 1: Factorial Calculation

This code defines a recursive function to calculate factorial of a number, where function repeatedly calls itself with smaller values until it reaches the base case.

```
In [67]: def factorial(n):  
        if n == 0: # Base case  
            return 1  
        else:      # Recursive case  
            return n * factorial(n - 1)  
  
print(factorial(5))
```

120

Explanation:

- Base Case: When $n == 0$, recursion stops and returns 1.
- Recursive Case: Multiplies n with the factorial of $n-1$ until it reaches the base case.

Example 2: Fibonacci Sequence

This code defines a recursive function to calculate n th Fibonacci number, where each number is the sum of the two preceding ones, starting from 0 and 1.

```
In [68]: def fibonacci(n):  
         if n == 0:  
             return 0  
         elif n == 1:  
             return 1  
         else:  
             return fibonacci(n-1) + fibonacci(n-2)  
  
         print(fibonacci(10))
```

55

Explanation:

- Base Cases: If $n == 0$, the function returns 0. If $n == 1$, the function returns 1. These two cases are necessary to stop the recursion.
- Recursive Case: function calls itself twice with decrements of n (i.e., $\text{fibonacci}(n-1)$ and $\text{fibonacci}(n-2)$), summing results of these calls.

Types of Recursion in Python

Recursion can be broadly classified into two types: tail recursion and non-tail recursion. The main difference between them is related to what happens after recursive call.

- Tail Recursion: The recursive call is the last thing the function does, so nothing happens after it returns. Some languages can optimize this to work like a loop, saving memory.
- Non-Tail Recursion: The function does more work after the recursive call returns, so it can't be optimized into a loop.

This code compares tail recursion and non-tail recursion using two versions of factorial function one with an accumulator (tail-recursive) and one with multiplication after recursive call (non-tail-recursive).

```
In [70]: def tail_fact(n, acc=1):
# Base case
if n == 0:
    return acc
# Tail recursive call with an accumulator
else:
    return tail_fact(n-1, acc * n)

def nontail_fact(n):
# Base case
if n == 1:
    return 1
# Non-tail recursive call because the multiplication happens after the call
else:
    return n * nontail_fact(n-1)

# Example usage
print(tail_fact(5))
print(nontail_fact(5))
```

```
120
120
```

Explanation:

- `def tail_fact(n, acc=1):` - Defines a tail-recursive factorial function with an accumulator `acc` to store intermediate results.
- `if n == 0: return acc` - Base case: when `n` reaches 0, return the accumulated result.
- `return tail_fact(n-1, acc * n)` - Tail-recursive call: multiplies `acc` by `n` before the call, so no extra work is left after recursion.
- `def nontail_fact(n):` - Defines a non-tail-recursive factorial function.
- `if n == 1: return 1` - Base case: factorial of 1 is 1.
- `return n * nontail_fact(n-1)` - Non-tail call: multiplication happens after the recursive call returns, so more work remains after recursion.

Recursion vs Iteration

Recursion:

It is often more intuitive and easier to implement when the problem is naturally recursive, like tree traversals. It can lead to solutions that are easier to understand compared to iterative ones.

Iteration:

Iteration involves loops (`for`, `while`) to repeat the execution of a block of code. It is generally more memory-efficient as it does not involve multiple stack frames like recursion.

When to Avoid Recursion

- When the problem can be solved easily with loops.
- When recursion depth is large enough to risk a stack overflow.
- When performance is critical and function call overhead matters.

11. Python Lambda Functions

Lambda Functions are anonymous functions means that the function is without a name. As we already know def keyword is used to define a normal function in Python. Similarly, lambda keyword is used to define an anonymous function in Python.

In the example, we defined a lambda function (upper) to convert a string to its upper case using upper().

```
In [71]: s1 = 'GeeksforGeeks'
s2 = lambda func: func.upper()
print(s2(s1))
```

GEEKSFORGEEKS

Explanation: s2 is a lambda function that takes a string and returns it in uppercase. Applying it to 'GeeksforGeeks' gives the result.

Syntax

lambda arguments : expression

- lambda: The keyword to define the function.
- arguments: A comma-separated list of input parameters (like in a regular function).
- expression: A single expression that is evaluated and returned.

Use Cases of Lambda Functions

Let's see some of the practical uses of the Python lambda function.

1. Using with Condition Checking

A lambda function can include conditions using if statements.

Here, the lambda function uses nested if-else logic to classify numbers as Positive, Negative or Zero.

```
In [73]: n = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
print(n(5))
print(n(-3))
print(n(0))
```

Positive
Negative
Zero

Explanation:

- The lambda function takes x as input.
- It uses nested if-else statements to return "Positive," "Negative," or "Zero."

This lambda checks divisibility by 2 and returns "Even" or "Odd" accordingly.

```
In [74]: check = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check(4))
print(check(7))
```

Even
Odd

Explanation:

- The lambda checks if a number is divisible by 2 ($x \% 2 == 0$).
- Returns "Even" for true and "Odd" otherwise.
- This approach is useful for labeling or categorizing values based on simple conditions.

2. Using with List Comprehension

Combining lambda with list comprehensions enables us to apply transformations to data in a concise way.

This code creates a list of lambda functions, each multiplying its input by 10 and then executes them one by one.

```
In [75]: li = [lambda arg=x: arg * 10 for x in range(1, 5)]
for i in li:
    print(i())
```

10
20
30
40

Explanation:

- The lambda function multiplies each element by 10.
- The list comprehension iterates through li and applies the lambda to each element.
- This is ideal for applying transformations to datasets in data preprocessing or manipulation tasks.

3. Using for Returning Multiple Results

Lambda functions do not allow multiple statements, however, we can create two lambda functions and then call the other lambda function as a parameter to the first function.

The lambda calculates both sum and product of two numbers and returns them as a tuple.

```
In [76]: calc = lambda x, y: (x + y, x * y)
res = calc(3, 4)
print(res)
```

(7, 12)

Explanation:

- The lambda function performs both addition and multiplication and returns a tuple with both results.
- This is useful for scenarios where multiple calculations need to be performed and returned together.

4. Using with filter()

The filter() function in Python takes in a function and a list as arguments. This offers an elegant way to filter out all the elements of a sequence "sequence", for which the function returns True.

Here, the lambda is used as a filtering condition to keep only even numbers from the list.

```
In [2]: n = [1, 2, 3, 4, 5, 6]
even = filter(lambda x: x % 2 == 0, n)
print(list(even))
```

```
[2, 4, 6]
```

Explanation:

- The lambda function checks if a number is even ($x \% 2 == 0$).
- `filter()` applies this condition to each element in `nums`.

5. Using with `map()`

The `map()` function in Python takes in a function and a list as an argument. The function is called with a lambda function and a new list is returned which contains all the lambda-modified items returned by that function for each item.

This code doubles each element of the list using a lambda function and returns a new list.

```
In [1]: a = [1, 2, 3, 4]
b = map(lambda x: x * 2, a)
print(list(b))
```

```
[2, 4, 6, 8]
```

Explanation:

- The lambda function doubles each number.
- `map()` iterates through `a` and applies the transformation.

6. Using with `reduce()`

The `reduce()` function in Python takes in a function and a list as an argument. The function is called with a lambda function and an iterable and a new reduced result is returned. This performs a repetitive operation over the pairs of the iterable. The `reduce()` function belongs to the `functools` module.

Here, the lambda multiplies two numbers at a time and `reduce()` applies this across the whole list to calculate the product.

```
In [79]: from functools import reduce
a = [1, 2, 3, 4]
b = reduce(lambda x, y: x * y, a)
print(b)
```

```
24
```

Explanation:

- The lambda multiplies two numbers at a time.
- `reduce()` applies this operation across the list.

Difference Between `lambda` and `def` Keyword

In Python, both lambda and def can be used to define functions, but they serve slightly different purposes.

```
In [80]: # Using Lambda
sq = lambda x: x ** 2
print(sq(3))

# Using def
def sqdef(x):
    return x ** 2
print(sqdef(3))
```

9
9

Explanation:

- lambda function sq takes a number and returns its square.
- regular function sqdef does the same but is defined using the def keyword.
- Both approaches give the same result, but lambda is more concise.

Now, let’s see a comparison between these two in tabular form:

Feature	lambda Function	Regular Function (def)
Definition	Single expression with lambda.	Multiple lines of code.
Name	Anonymous (or named if assigned).	Must have a name.
Statements	Single expression only.	Can include multiple statements.
Documentation	Cannot have a docstring.	Can include docstrings.
Reusability	Best for short, temporary functions.	Better for reusable and complex logic.

Kamran Khan